

TensorBinding.jl

Package Function Reference

April 25, 2026

Contents

1	Overview	3
2	Central Data Structure: TBHamiltonian	3
3	Hamiltonian Construction (TBSystem.jl)	4
3.1	Constructor	4
3.2	Cache and Truncation Helpers	5
3.3	Incremental Hamiltonian Modification	5
3.4	Geometry Helpers	6
4	Low-Level MPO Builders (Hamiltonian.jl, 2D_lattice.jl)	6
4.1	1D Kinetic Operators	6
4.2	2D Kinetic Operators	7
4.3	Diagonal Mask MPOs	7
4.4	QTCI-Based Builders	8
4.5	Preset Model Hamiltonians (direct MPO)	8
5	Bilayer and Multilayer Hamiltonians	9
5.1	Commensurate Stacking (bilayer_tk.jl)	9
5.2	Twisted Multilayer (twisted_tk.jl)	9
5.3	Layer-Core Prepend/Postpend Helpers	10
6	Spin and Nambu Extensions (Supercond_tk.jl)	10
6.1	Index Constructors	10
6.2	Unified Prepend/Postpend Operators (utils.jl)	10
6.3	Spin and Nambu Wrappers	10
6.4	Higher-Level Assemblers	11
7	Kernel Polynomial Method (KPM_tk.jl)	11
7.1	Spectral Bounds Estimation	11
7.2	Building the Chebyshev List	11
7.3	Spectral Observables	12
7.4	HODC Kernel	12
8	Density Matrix Purification (Purification_tk.jl)	12
8.1	Initial Guess	12
8.2	McWeeny Purification	12
8.3	SP2 Purification	13

9 Quantum Fourier Transform and Band Structure (QFT_tk.jl)	13
10 Topological Invariants (Topology_tk.jl)	13
10.1 Projector Retrieval	14
10.2 Quenched Position Operators	14
10.3 1D Winding Number	14
10.4 2D Real-Space Chern Marker	15
11 DMRG (DMRG_tk.jl)	15
12 Time Evolution (Timeev_tk.jl)	15
13 Random Phase Approximation (RPA_tk.jl)	16
13.1 Pipeline	16
13.2 Functions	16
13.3 Tensor Utilities (used internally by RPA)	16
14 Self-Consistent Mean Field (Meanfi_tk.jl)	17
15 Utilities (utils.jl)	17
16 Custom Operator Extensions	17
17 Quick-Reference Workflows	18
17.1 KPM Density of States and Band Structure	18
17.2 Density Matrix Purification	19
17.3 Topological Invariants	19
17.4 Bilayer and Twisted Multilayer	19
17.5 Spinful Superconductor (BdG)	19
17.6 aux_side: postpending auxiliary DOF	20

Overview

TensorBinding.jl is a Julia package for constructing and manipulating tight-binding Hamiltonians as Matrix Product Operators (MPOs) in the *quantics binary* (QTT) representation. A system of $N = 2^L$ sites is encoded in L qubit sites: site index $n \in \{0, \dots, N-1\}$ is identified with the binary string $b_1 b_2 \dots b_L$. This representation makes the bond dimension of many physically relevant Hamiltonians small (typically ≤ 10), enabling compression via Quantics Tensor Cross Interpolation (QTCI).

Key capabilities:

- MPO construction for 1D and 2D lattices (square, triangular, honeycomb) via shift operators and QTCI.
- Bilayer, multilayer, twisted, and Bernal-stacked Hamiltonians.
- Spin- $\frac{1}{2}$ and Nambu (BdG) extensions via a prepend/postpend-core pattern, with auxiliary DOF placeable at front (`:pre`) or back (`:post`).
- Kernel Polynomial Method (KPM) for density of states, density matrices, spectral functions, and Green's functions.
- Density-matrix purification (McWeeny, SP2) with `TBHamiltonian`-aware API.
- Quantum Fourier Transform conjugation for band structure $A(k, \omega)$, supporting spinful, BdG, and layered Hamiltonians via `aux_proj`.
- Topological invariants: real-space Chern marker and winding-number density via quenched or flat position operators, compatible with any lattice geometry.
- DMRG ground state and spectral DMRG.
- Real-time evolution: TDVP (fixed and time-dependent H), propagator MPO.
- Random Phase Approximation (RPA) for susceptibility.
- Self-consistent mean-field (Hubbard model).

Site encoding conventions:

- *Position qubits*: L sites, site i (0-indexed) $\leftrightarrow$ binary string $b_1 \dots b_L$, big-endian.
- *Extended systems*: auxiliary indices can be *prepended* (`:pre`, default) or *postpended* (`:post`). For a spinful bilayer BdG system with `:pre` the site chain is `[nambu_s, spin_s, layer_s, pos_qubits...]`.
- *2D lattices*: row-major flattening, linear index $i = i_x + i_y \cdot 2^{L_x}$, $i_x \in \{0, \dots, 2^{L_x} - 1\}$, $i_y \in \{0, \dots, 2^{L_y} - 1\}$.
- *Honeycomb*: intra-row bonds pair even and odd i_x sites within each row; inter-row bonds have two inequivalent types controlled by sublattice masks. Requires $L_x + L_y$ total qubits and $L = L_x + L_y$.

Central Data Structure: TBHamiltonian

Defined in `src/TBSystem.jl`.

Field	Type	Description
<code>L</code>	<code>Int</code>	Number of position qubits; $N = 2^L$ sites.
<code>N</code>	<code>Int</code>	Number of physical sites per spin/Nambu/layer sector.
<code>sites</code>	<code>Vector{<:Index}</code>	All ITensor site indices (includes auxiliary indices if present).
<code>mpo</code>	<code>MPO</code>	The accumulated Hamiltonian MPO.
<code>geometry</code>	<code>Function</code> or <code>Nothing</code>	Callable <code>i -> pos_vec</code> (1-indexed); returns real-space position of site <code>i</code> .
<code>scale</code>	<code>Float64</code>	Energy half-bandwidth; <code>H/scale</code> has spectrum $\subseteq [-1, 1]$. Set to 0 for lazy DMRG estimation.
<code>center</code>	<code>Float64</code>	Spectral center $(E_{\max} + E_{\min})/2$. 0 for particle-hole symmetric spectra.
<code>spin_s</code>	<code>Index</code> or <code>Nothing</code>	Spin- $\frac{1}{2}$ index (set by <code>add_spin!</code>).
<code>nambu_s</code>	<code>Index</code> or <code>Nothing</code>	Nambu index (set by <code>add_superconductivity!</code>).
<code>layer_s</code>	<code>Index</code> or <code>Nothing</code>	Layer index (set by bilayer/multilayer constructors).
<code>aux_side</code>	<code>Symbol</code>	<code>:pre</code> (aux at front, default) or <code>:post</code> (aux at back).
<code>_tn_cache</code>	<code>Vector{MPO}</code> or <code>Nothing</code>	Cached Chebyshev list $T_0 \dots T_N$.
<code>_tn_Ncheb</code>	<code>Int</code>	Order N of cached Chebyshev list.
<code>_density_cache</code>	<code>MPO</code> or <code>Nothing</code>	Cached density matrix from purification.

Do not construct directly; use `get_Hamiltonian` or a layer/multilayer constructor. Caches are invalidated automatically by all `add_*` functions and `truncate!`. The `center` field is set together with `scale` during spectral-bounds estimation and is needed to correctly rescale energies: $\tilde{\omega} = (\omega - \text{center})/\text{scale}$.

Hamiltonian Construction (TBSystem.jl)

Constructor

Signature	Description
<code>get_Hamiltonian(geometry, params; L, scale, tol, maxdim, kwargs)</code>	Build a <code>TBHamiltonian</code> from a named model. <code>geometry</code> is a string key; <code>params</code> holds model parameters (see below). Returns a <code>TBHamiltonian</code> .

Supported geometry strings:

geometry	params	Notes
"chain_1d"	hopping t	1D NN chain, direct MPO (no QTCI).
"square_2d"	hopping t	2D square, $2^{L_x} \times 2^{L_y}$ sites; Lx/Ly kwargs.
"hex_2d"	hopping t	2D honeycomb.
"triangular_2d"	hopping t	2D triangular.
"ssh"	(t, d)	SSH chain: hopping $t \pm d$ on alternating bonds.
"aah"	(V, ϕ, t)	Aubry–André–Harper quasicrystal.
"uniform"	t	Uniform 1D with optional on-site v .
"haldane"	(t_2, ϕ, M) NamedTuple	Haldane model; requires rs ($N \times 2$ position matrix).
"chernhex"	(t, t_2, ms)	Haldane-type Chern insulator on honeycomb.
"chern8"	(V, t)	8-fold Chern mosaic on square lattice.
"qc2dsquare"	t	Quasicrystal-modulated square lattice.
"custom"	$f(i, j)$	Arbitrary hopping matrix; scale required.

Cache and Truncation Helpers

Signature	Description
<code>_invalidate_cache!(H) → H</code>	Clear all cached results (<code>_tn_cache</code> , <code>_density_cache</code> , <code>scale/center</code>). Called automatically by all <code>add_*!</code> functions.
<code>truncate!(H; cutoff=1e-10, maxdim=nothing) → H</code>	Truncate <code>H.mpo</code> in-place; invalidates all caches.

Incremental Hamiltonian Modification

All functions below mutate `H` in place and invalidate the Chebyshev cache. `add_hopping!` and `add_onsite!` must be called before `add_spin!` or `add_superconductivity!`.

The optional `position` keyword (default: inherits `H.aux_side`) controls whether auxiliary indices are prepended (`:pre`) or postpended (`:post`).

Signature	Description
<code>add_hopping!(H, f; maxdim=15, tol=1e-8, type=ComplexF64)</code>	Add an arbitrary hopping $f(i, j)$ to <code>H.mpo</code> via QTCI. Errors if any auxiliary index is already present.
<code>add_onsite!(H, f; tol=1e-8)</code>	Add a diagonal on-site term $f(i)$ (1-indexed) via <code>get_diagonal_mpo</code> .
<code>add_spin!(H; cutoff=1e-8, maxdim=200, position=:pre)</code>	Prepend (or postpend) a spin- $\frac{1}{2}$ index: $\mathbf{I}_{\text{spin}} \otimes H$. No-op if already spinful.

<code>add_zeeman!(H, h; direction=:z, tol=1e-8, maxdim=200, position)</code>	Add Zeeman coupling $h \cdot S_\alpha$. <code>h</code> is a Number or function $h(i)$. <code>direction</code> $\in \{ :x, :y, :z \}$. Calls <code>add_spin!</code> automatically. If BdG is already applied, wraps in τ_z .
<code>add_superconductivity!(H, Δ; type=:swave, tol=1e-8, maxdim=200, position)</code>	BdG extension: prepend (or postpend) Nambu index. Spinless or singlet ($i\sigma_y \otimes \Delta$) depending on whether spin is present. <code>type=:swave</code> : $\Delta(i)$ diagonal; <code>type=:custom</code> : $\Delta(i, j)$ via TCI. Errors if called twice.
<code>add_soc!(H, λ; type=:rashba, direction=:z, tol=1e-8, maxdim=200, position)</code>	Spin-orbit coupling. <code>type=:rashba</code> : NN Rashba $\lambda(S_y K_u - S_y K_d)$ (scalar λ only). <code>type=:ising</code> : $\lambda(i) \cdot S_z$. <code>type=:custom</code> : arbitrary position MPO $\times$ spin operator given by <code>direction</code> .

Typical workflow:

```
H = get_Hamiltonian("square_2d", 1.0; L=8)
add_onsite!(H, _ -> -0.5)           # chemical potential
add_zeeman!(H, 0.1)                  # Zeeman splits spin sectors
add_soc!(H, 0.05)                    # Rashba SOC
add_superconductivity!(H, 0.2)       # s-wave BdG
# H.sites = [nambu_s, spin_s, pos_qubits...] (default :pre stacking)
```

Geometry Helpers

Signature	Description
<code>honeycomb_positions(L; Lx=L÷2) → Matrix{Float64}</code>	Return $N \times 2$ real-space positions for a honeycomb patch ($N = 2^L$).
<code>square_positions(L; Lx=L÷2) → Matrix{Float64}</code>	Return $N \times 2$ real-space positions for a square lattice patch.
<code>triangular_positions(L; Lx=L÷2) → Matrix{Float64}</code>	Return $N \times 2$ real-space positions for a triangular lattice patch. Odd rows offset by 0.5 in x ; $y = i_y \cdot \sqrt{3}/2$.
<code>lattice_positions(lattice, Lx, Ly; angle_deg=0.0) → Matrix{Float64}</code>	Positions for <code>:square</code> , <code>:triangular</code> , or <code>:honeycomb</code> , optionally rotated by <code>angle_deg</code> degrees about the centroid. (<code>twisted_tk.jl</code>)
<code>central_index(geom, N) → Int</code> <code>central_index(H) → Int</code>	1-indexed site whose position is closest to the geometric centroid. <code>geom</code> is a callable <code>i -> pos_vec</code> . Errors if <code>H.geometry</code> is <code>nothing</code> .
<code>_pos_sites(H) → Vector</code>	Last L (or first L if <code>aux_side=:post</code>) entries of <code>H.sites</code> : position qubits, skipping all auxiliary indices.

Low-Level MPO Builders (`Hamiltonian.jl`, `2D_lattice.jl`)

These functions build MPOs directly on a provided `sites` vector.

1D Kinetic Operators

Signature	Description
<code>kinetic_1d_nn(L, sites) → MPO</code>	Uniform NN hopping $\sum_i(i+1\rangle\langle i + \text{h.c.})$ via <code>sigma_plus/sigma_minus</code> binary increment/decrement operators.
<code>kinetic_1d_nn_custom(L, sites, hopping) → MPO</code>	Spatially varying NN hopping; <code>hopping</code> is a diagonal MPO of amplitudes.
<code>kineticNNN(L, sites, hopping, nn) → MPO</code>	1D chain with n -th-nearest-neighbour hopping and site-varying amplitude. <code>nn=1</code> = NN; larger <code>nn</code> built via <code>compose_power</code> .
<code>generate_kin_u(sites, N) → MPO</code>	Shift MPO $ n+1\rangle\langle n $ (binary increment / up-shift).
<code>generate_kin_d(sites, N) → MPO</code>	Shift MPO $ n-1\rangle\langle n $ (binary decrement / down-shift).

2D Kinetic Operators

Signature	Description
<code>intrachain_hopping(L_chain, N, sites; hopping=Id, t=1) → MPO</code>	NN x -direction hopping on a 2D lattice with row-break mask applied.
<code>interchain_hopping_square(L_chain, N, sites; hopping=Id, t=1) → MPO</code>	NN y -direction hopping on a square lattice.
<code>interchain_hopping_square_2nd_plus(L_chain, N, sites; hopping=Id, t2=1) → MPO</code>	NN hopping along $(+x, +y)$ diagonal (shift $+L_{\text{chain}} + 1$); row-break mask prevents wrap-around.
<code>interchain_hopping_square_2nd_minus(L_chain, N, sites; hopping=Id, t2=1) → MPO</code>	NN hopping along $(+x, -y)$ diagonal (shift $-(L_{\text{chain}} - 1)$).
<code>interchain_hopping_triangle(L_chain, N, sites) → MPO</code>	Both diagonal inter-row bonds on a triangular lattice (SW↗NE and SE↖NW).
<code>kineticintra2DNNN(Lx, Ly, sites, hopping, nn) → MPO</code>	Long-range intra-row hopping on $2^{L_x} \times 2^{L_y}$ square lattice. Row-boundary wrap suppressed by <code>_row_break_mpo</code> .
<code>kineticintra2DNNhex(Lx, Ly, sites, hopping, nn) → MPO</code>	Intra-row hopping for honeycomb; applies checkerboard A/B mask.
<code>kineticinterNNNSWNE(Lx, Ly, sites, hopping, nn) → MPO</code>	SW↗NE diagonal inter-row hopping (square lattice, <code>:xplus</code> mask).
<code>kineticinterNNNSENW(Lx, Ly, sites, hopping, nn) → MPO</code>	SE↖NW diagonal inter-row hopping (<code>:xplain</code> mask).
<code>kineticinterNNNtriSWNE(Lx, Ly, sites, hopping, nn) → MPO</code>	SW↗NE diagonal for triangular lattice; adds even-row selection mask.
<code>kineticinterNNNtriSEnw(Lx, Ly, sites, hopping, nn) → MPO</code>	SE↖NW diagonal for triangular lattice; adds odd-row selection mask.

Diagonal Mask MPOs

Used internally as connectivity filters; all return diagonal 0/1 MPOs.

Signature	Description
<code>_row_break_mpo(Lx, Ly, sites; which::Symbol) → MPO</code>	Mask 0 at row boundaries. <code>which=:xplus</code> : last column; <code>:xplain</code> : first column.

<code>_row_select_mpo(_, Ly, sites; keep=:even) → MPO</code>	Retain only even or odd rows (<code>keep=:even</code> or <code>:odd</code> , 0-based).
<code>_col_select_mpo(Lx, Ly, sites; keep=:even) → MPO</code>	Retain only even or odd columns.
<code>_row_checker_mpo(Lx, Ly, sites) → MPO</code>	Checkerboard: 1 where $(i_x + i_y)$ is even, 0 otherwise.
<code>skeleton(L_chain, N, sites) → MPO</code>	0 where $i_x = 1$ (second column, 0-indexed), 1 elsewhere. Used internally for triangular wrap-around suppression.

QTCI-Based Builders

Signature	Description
<code>hopping2MPO(f, N, sites; tol=1e-8, initial_positions=[], type=Float64, unfolding_scheme=:interleaved) → MPO</code>	Compress $N \times N$ matrix $H[i, j] = f(i, j)$ into an MPO via QTCI. <code>unfolding_scheme=:interleaved</code> (default) or <code>:fused</code> .
<code>qtc_matrix_to_MPO(A_fun, L, sites; tol=1e-8, type=Float64) → MPO</code>	Like <code>hopping2MPO</code> for $2^L \times 2^L$ matrix functions; applies extra truncation at <code>maxdim=20</code> .
<code>get_diagonal_mpo(L, sites, f; type=Float64) → MPO</code>	Diagonal MPO with entry $f(x)$ at site $x \in \{1, \dots, 2^L\}$ (1-indexed). Compressed via 1D QTCI.
<code>qtt_mpo(L, xvals, sites, func; tol=1e-8, maxbonddim=50) → MPO</code> <code>compose_power(base, nn; side=:right, apply_kwargs) → MPO</code>	Compress scalar function <code>func(x)</code> on grid <code>xvals</code> into diagonal MPO via QTCI. <code>baseⁿⁿ</code> by exponentiation-by-squaring in $O(\log n)$ applies. Returns identity if <code>nn=0</code> .

Preset Model Hamiltonians (direct MPO)

Function	Description
<code>HUniform(L, t; v=1e-6, nn=1)</code>	Uniform 1D chain with optional on-site potential <code>v</code> .
<code>HSSH(L, t, d; nn=1)</code>	SSH dimerized chain: hopping $t \pm d$ on alternating bonds.
<code>HAAH(L, V, phi, t; b=(1+$\sqrt{5}$)/2)</code>	Aubry–André–Harper: $H = -t \sum c_{i+1}^\dagger c_i + V \cos(2\pi b i + \phi) n_i$.
<code>HUniform2DSquare(Lx, Ly, t)</code>	Uniform square lattice tight-binding.
<code>HUniform2Dhex(Lx, Ly, t)</code>	Uniform honeycomb lattice (checkerboard intra-row mask).
<code>HUniform2Dtri(Lx, Ly, t)</code>	Uniform triangular lattice (three kinetic terms).
<code>HChern8(Lx, Ly, V, t)</code>	8-fold Chern mosaic with spatially-varying hopping.
<code>H2DChernhex(Lx, Ly, t, t2, ms)</code>	Haldane Chern insulator on honeycomb; domain walls in t_2 and Semenoff mass.
<code>HQC2DSquare(Lx, Ly, t)</code>	Quasicrystal-modulated square lattice (8-fold wavevector pattern).
<code>build_hamiltonian(model, L; mparams, mparam_dict)</code>	Dispatch via <code>MODEL_REGISTRY</code> : build 1D MPO from model name and parameter string.


```
build_hamiltonian(model,      2D variant.
Lx, Ly; mparams,
mparam_dict)
```

Known models in MODEL_REGISTRY: "uniform", "ssh", "aah", "square_2d", "hex_2d", "triangular_2d", "chern8", "chernhex", "qc2dsquare".

Bilayer and Multilayer Hamiltonians

Commensurate Stacking (bilayer_tk.jl)

Site encoding: site 1 = layer qubit (Index of dim n_{layers}); sites $2 \dots L + 1$ = position qubits. Constructors return a TBHamiltonian with H.layer_s set.

Signature	Description
bilayer_hamiltonian(lattice, Lx, Ly; stacking=:AA, t_intra=1.0, t_inter=0.3, cutoff=1e-8, maxdim=200) → TBHamiltonian	Two-layer Hamiltonian: $\sum_k P_k \otimes H_{\text{mono}} + (1\rangle\langle 2 + 2\rangle\langle 1) \otimes V$. stacking=:AA (on-site) or :Bernal (A_1 – B_2 , honeycomb only).
multilayer_hamiltonian(lattice, Lx, Ly, n_layers; stacking=:AA, t_intra=1.0, t_inter=0.3, cutoff=1e-8, maxdim=200) → TBHamiltonian	Generalises to n_{layers} layers; interlayer coupling between adjacent pairs only.
interlayer_mpo(lattice, stacking, Lx, Ly, sites; t_inter=1.0, cutoff=1e-8) → MPO	Build the position-space interlayer coupling MPO V . Routes to _bernal_interlayer_mpo or _aa_interlayer_mpo .
_bernal_interlayer_mpo(L, sites; t_inter=1.0, cutoff=1e-8) → MPO	Exact Bernal A_1 – B_2 coupling: $V = t_{\perp}(K_u D_A + D_A K_d)$. D_A projects onto odd (A) sites (1-indexed); $K_{u/d}$ are ± 1 shifts.
_aa_interlayer_mpo(sites; t_inter=1.0) → MPO	AA stacking: on-site coupling $V = t_{\perp} \mathbf{I}$.

Twisted Multilayer (twisted_tk.jl)

Signature	Description
twisted_multilayer_hamiltonian(lattice, Lx, Ly, angles_deg; t_intra=1.0, t_inter=0.3, $\alpha_{\text{decay}}=1/16$, tol=1e-6, cutoff=1e-8, maxdim=200) → TBHamiltonian	Twisted multilayer: layer k rotated to angles_deg[k] . Interlayer coupling $V_{kl}(i, j) = t_{\perp} e^{-\alpha \mathbf{r}_k^{(i)} - \mathbf{r}_l^{(j)} }$ compressed via QTCI.
twisted_bilayer_hamiltonian(lattice, Lx, Ly, θ_{deg} ; ...) → TBHamiltonian	Convenience wrapper: layer 1 at 0° , layer 2 at θ_{deg} .
monolayer_hamiltonian(lattice, Lx, Ly, sites; t=1.0, cutoff=1e-8) → MPO	Single-layer NN Hamiltonian MPO for :square , :triangular , or :honeycomb .

Layer-Core Prepend/Postpend Helpers

These functions extend an L -site MPO by one site by prepending or postpending a single-site layer operator. All are thin wrappers around `prepend_op/postpend_op`.

Signature	Description
<code>prepend_layer_projector(H, layer_s, k) → MPO</code>	Prepend $ k\rangle\langle k $ (1-indexed). Used for $P_k \otimes H$.
<code>prepend_layer_hopping(H, layer_s, k, l) → MPO</code>	Prepend $ k\rangle\langle l $. Used for interlayer $ k\rangle\langle l \otimes V$.
<code>postpend_layer_projector(H, layer_s, k) → MPO</code>	Postpend $ k\rangle\langle k $ to the end of H.
<code>postpend_layer_hopping(H, layer_s, k, l) → MPO</code>	Postpend $ k\rangle\langle l $ to the end of H.

Spin and Nambu Extensions (Supercond_tk.jl)

Index Constructors

Function	Description
<code>spin_index()</code>	Create a dim-2 Index tagged "Spin" (1= $\uparrow$, 2= $\downarrow$).
<code>nambu_index()</code>	Create a dim-2 Index tagged "Nambu" (1=particle, 2=hole).

Unified Prepend/Postpend Operators (utils.jl)

Signature	Description
<code>prepend_op(H, s, mat) → MPO</code>	General prepend: extends H by prepending an arbitrary matrix <code>mat</code> on site <code>s</code> .
<code>prepend_op(H, s, op::Symbol) → MPO</code>	Named operator lookup for "Spin" or "Nambu" tagged indices.
<code>prepend_op(H, s, k, l) → MPO</code>	Sparse $ k\rangle\langle l $ on site <code>s</code> (any dimension; 1-indexed).
<code>prepend_op(H, s, k) → MPO</code>	Projector $ k\rangle\langle k $.
<code>postpend_op(H, s, mat / op / k, l / k) → MPO</code>	Symmetric counterpart of <code>prepend_op</code> ; appends to the end of H.

Spin and Nambu Wrappers

Signature	Description
<code>prepend_spin(H, spin_s, op) → MPO</code>	Prepend a spin- $\frac{1}{2}$ operator. <code>op</code> is a <code>Symbol</code> or 2×2 matrix. Built-in ops: <code>:Id</code> , <code>:Pup</code> , <code>:Pdn</code> , <code>:Sz</code> , <code>:Sp</code> , <code>:Sm</code> , <code>:Sx</code> , <code>:Sy</code> , <code>:iSy</code> , <code>:miSy</code> .
<code>postpend_spin(H, spin_s, op) → MPO</code>	Postpend a spin- $\frac{1}{2}$ operator to the end of H. Same ops as above.
<code>prepend_nambu(H, nambu_s, op) → MPO</code>	Prepend a Nambu (particle-hole) operator. Built-in ops: <code>:Id</code> , <code>:Pp</code> , <code>:Ph</code> , <code>:tz</code> , <code>:tx</code> , <code>:ty</code> , <code>:tp</code> , <code>:tm</code> .

<code>postpend_nambu(H, nambu_s, op) → MPO</code>	Postpend a Nambu operator to the end of H. Same ops as above.
---	--

Higher-Level Assemblers

Signature	Description
<code>spin_hamiltonian(H_up, H_dn, spin_s; H_Zeeman=nothing, cutoff=1e-8) → MPO</code>	$P_{\uparrow} \otimes H_{\uparrow} + P_{\downarrow} \otimes H_{\downarrow} [+S_z \otimes H_{\text{Zeeman}}]$.
<code>bdg_hamiltonian(H_kin, H_pair, nambu_s; cutoff=1e-8) → MPO</code>	Spinless BdG: $\tau_z \otimes H_{\text{kin}} + \tau_+ \otimes H_{\text{pair}} + \tau_- \otimes H_{\text{pair}}^{\dagger}$.
<code>bdg_spin_hamiltonian(H_up, H_dn, H_pair, spin_s, nambu_s; H_soc=nothing, cutoff=1e-8) → MPO</code>	Spin- $\frac{1}{2}$ singlet BdG in Nambu-spin space. Pairing: $\tau_+ \otimes (i\sigma_y) \otimes H_{\text{pair}} + \text{h.c.}$. Optional Ising SOC $\tau_z \otimes S_z \otimes H_{\text{soc}}$.

Kernel Polynomial Method (KPM.tk.jl)

The KPM expands spectral quantities in Chebyshev polynomials $T_n(\tilde{H})$ where $\tilde{H} = (H - \text{center})/\text{scale} \in [-1, 1]$. Supported kernels: `:jackson` (default), `:lorentz`, `:fejer`, `:dirichlet`, `:hodc`.

Spectral Bounds Estimation

Signature	Description
<code>_estimate_spectral_bounds(H_mpo, sites; dmrg_nsweeps=5, dmrg_maxdim=[10,20,40], dmrg_linkdim=4) → (scale, center)</code> <code>_ensure_scale!(H::TBHamiltonian; ...)</code>	Run DMRG to minimise H and $-H$, yielding $E_{\min}/E_{\max}$. Returns <code>scale</code> = $(E_{\max} - E_{\min})/2 \times 1.1$ and <code>center</code> = $(E_{\max} + E_{\min})/2$. If $H.\text{scale} == 0$, call <code>_estimate_spectral_bounds</code> and store in $H.\text{scale}/H.\text{center}$. No-op otherwise. Called lazily by all high-level KPM functions.

Building the Chebyshev List

Signature	Description
<code>KPM_Tn(H_mpo, N, sites; scale=nothing, center=0.0, maxdim=40, cutoff=1e-8, ...) → (Tn_list, scale, center)</code>	Low-level form. Build Chebyshev MPOs $T_n(\tilde{H})$ for $n = 0 \dots N$. If <code>scale=nothing</code> , bounds estimated via DMRG.
<code>KPM_Tn(H::TBHamiltonian, Ncheb; maxdim=40, cutoff=1e-8, ...) → (Tn_list, scale, center)</code>	High-level form. Calls <code>_ensure_scale!</code> lazily. Caches result in $H._\text{tn_cache} / H._\text{tn_Ncheb}$.

Important — return order: The return tuple is `(Tn_list, scale, center)`. A common bug is `Tn, _, scale = KPM_Tn(...)`, which assigns `center` to `scale`. The correct pattern is:

```
Tn, scale, center = KPM_Tn(H, Ncheb, sites)
omega_r = (omega - center) / scale # rescaled energy in (-1, 1)
```

When using the TBHamiltonian overload, `H.scale` and `H.center` are updated automatically and can be read directly.

Spectral Observables

Signature	Description
<code>get_ldos_w_from_Tn(Tn_list, N, ω; maxdim=40, kernel=:jackson, ...) → MPO</code>	Spectral MPO $\delta(\omega - H)$ at rescaled energy $\omega \in (-1, 1)$. $\text{DoS}(\omega) = \text{Tr}[\delta(\omega - H)]$.
<code>get_Green_retarded_from_Tn(Tn_list, N, ω; $\eta=1\text{e-}2$, maxdim=40, kernel=:jackson, ...) → MPO</code>	Retarded Green's function MPO $G^R(\omega + i\eta) = (\omega + i\eta - H)^{-1}$.
<code>get_density_from_Tn(Tn_list, N; fermi=0.0, maxdim=40, kernel=:jackson, ...) → MPO</code>	Density matrix $\rho = \theta(\mu - H)$ at rescaled Fermi level <code>fermi</code> .
<code>get_PH_from_Tn(Tn_list, N, ω; maxdim=40, kernel=:jackson, ...) → MPO</code>	Particle-hole spectral function MPO.

HODC Kernel

Signature	Description
<code>compute_hodc_params(m=6) → (z1, w1)</code>	Compute m quadrature nodes <code>z1</code> and weights <code>w1</code> for the HODC kernel. Call once; pass to KPM functions as <code>kernel=:hodc</code> , <code>z1=z1</code> , <code>w1=w1</code> .

Density Matrix Purification (Purification.tk.jl)

Iterative methods that drive eigenvalues of an approximate density matrix to exactly 0 or 1, refining the zero-temperature projector $\theta(\mu - H)$ without Gibbs ringing.

Initial Guess

Signature	Description
<code>purification.initial_guess(H.mpo, scale, sites; maxdim=40, cutoff=1e-8) → MPO</code>	Low-level form. Build $\rho_0 = \frac{1}{2}I - \frac{1}{2s}H$. Works only for center-symmetric spectra.
<code>purification.initial_guess(H::TBHamiltonian, maxdim=40, cutoff=1e-8) → MPO</code>	High-level form. Calls <code>_ensure_scale!</code> automatically. Uses formula accounting for nonzero center: $\rho_0 = (\frac{1}{2} + \frac{\text{center}}{2\text{scale}})I - \frac{H}{2\text{scale}}$.

McWeeny Purification

Signature	Description
-----------	-------------

<code>mcweeny_purify(ρ_0; maxiters=30, maxdim=40, cutoff=1e-8, tol=1e-5, verbose=false) → MPO</code>	Iterate $\rho_{n+1} = 3\rho_n^2 - 2\rho_n^3$ until $\ \rho^2 - \rho\ /\ \rho\ < \text{tol}$. Quadratic convergence; 2 MPO products per step.
<code>mcweeny_purify(H::TBHamiltonian; maxiters=30, maxdim=40, cutoff=1e-8, tol=1e-5, verbose=false) → MPO</code>	High-level form. Builds initial guess from H, runs McWeeny, caches result in <code>H._density_cache</code> .

SP2 Purification

Signature	Description
<code>sp2_purify(ρ_0, Nel; maxiters=40, maxdim=40, cutoff=1e-8, tol=1e-5, verbose=false) → MPO</code>	Iterate: if $\text{Tr}(\rho^2) \geq N_{\text{el}}$: $\rho_{n+1} = \rho^2$; else $\rho_{n+1} = 2\rho - \rho^2$. Drives $\text{Tr}(\rho) \rightarrow N_{\text{el}}$; 1 MPO product per step.
<code>sp2_purify(H::TBHamiltonian; Nel=H.N÷2, maxiters=40, maxdim=40, cutoff=1e-8, tol=1e-5, verbose=false) → MPO</code>	High-level form. Nel defaults to half-filling. Caches result in <code>H._density_cache</code> .

Quantum Fourier Transform and Band Structure (QFT_tk.jl)

Signature	Description
<code>conjugate_by_qft(W; tol=1e-9, maxdim=100) → MPO</code>	Returns $UWU^\dagger$ where U is the QFT MPO. Maps any real-space MPO to momentum space.
<code>get_spect_k(W; tol=1e-9, maxdim=100) → Vector</code>	Momentum-space diagonal $\langle k \tilde{W} k\rangle$ for a single-band 1D system. Returns length- N vector.
<code>get_spect_k_doubled(W; tol=1e-9, maxdim=100) → Vector</code>	Same for a doubled ($2L$ -site) system (two-band or spin-resolved).
<code>get_bands(Tn_list, Ncheb, sites, ω_vals; tol=1e-9, maxdim=100) → Matrix{Float64}</code>	Low-level form. Compute $A(k, \omega) = \langle k U\delta(\omega - H)U^\dagger k\rangle$. Returns $(N_k \times N_\omega)$ matrix. ω _vals must be in <i>rescaled</i> units $\in (-1, 1)$.
<code>get_bands(H::TBHamiltonian, ω_phys_vals; aux_proj=nothing, tol=1e-9, maxdim=100) → Matrix{Float64}</code>	High-level form. Works for plain, spinful, BdG, and layered Hamiltonians. The QFT acts only on position qubits; identity is prepended/postpended on aux sites. aux_proj=nothing : traces over aux DOF. aux_proj=[$\sigma_1, \dots$] : fixes each aux site to the given 1-based state. Requires a prior <code>KPM.Tn</code> call to populate <code>H._tn_cache</code> .

Topological Invariants (Topology_tk.jl)

Real-space topological invariants (Chern number and winding number) for general lattice geometries. All functions accept user-supplied coordinate functions `xfunc(i, L_chain)` and `yfunc(i, L_chain)` where `i` is a 0-indexed site number and `L_chain=2l` is the relevant lattice length.

Key concept — quenching: Periodic boundary conditions would make $\hat{x}$ discontinuous. With `quenched=true` the position operators are replaced by $\sin(x/\Lambda)$ and $\cos(x/\Lambda)$, which are

bounded and smooth. The identity $x \approx \Lambda \sin(x/\Lambda)$ for small arguments means the quenched Chern marker recovers physical units via a factor Λ^2 . The winding-number density recovers via Λ .

Projector Retrieval

Signature	Description
<code>_get_projector(H; method=:KPM, fermi=0.0, Nchebychev=300, maxdim=40, cutoff=1e-8, Nel=nothing) → MPO</code>	Compute or retrieve the zero-temperature projector $P = \theta(\mu - H)$. <code>method=:KPM</code> : uses cached <code>H._tn_cache</code> or runs KPM; <code>fermi</code> in physical units. <code>method=:mcweeny</code> : uses <code>H._density_cache</code> or calls <code>mcweeny_purify</code> . <code>method=:sp2</code> : uses <code>H._density_cache</code> or calls <code>sp2_purify</code> with target <code>Nel</code> .

Quenched Position Operators

These build diagonal MPOs for the trigonometric quenching functions. The argument to `get_diagonal_mpo` is 1-indexed; these builders convert internally with `xfunc(i-1, L_chain)`.

Signature	Description
<code>get_sinx_op(L, sites, L_chain, Λ, xfunc) → MPO</code>	Diagonal MPO for $\sin(x_i/\Lambda)$.
<code>get_cosx_op(L, sites, L_chain, Λ, xfunc) → MPO</code>	Diagonal MPO for $\cos(x_i/\Lambda)$.
<code>get_siny_op(L, sites, L_chain, Λ, yfunc) → MPO</code>	Diagonal MPO for $\sin(y_i/\Lambda)$.
<code>get_cocy_op(L, sites, L_chain, Λ, yfunc) → MPO</code>	Diagonal MPO for $\cos(y_i/\Lambda)$.

1D Winding Number

Signature	Description
<code>get_W(H, xfunc, sz_func; method=:KPM, fermi=0.0, Nchebychev=300, maxdim=15, cutoff=1e-8, Nel=nothing, quenched=true, l=nothing, $\Lambda=10$) → Function</code>	Compute the winding-number density operator. Pre-computes one or two MPOs (W_1, W_2 for <code>quenched=true</code>). Returns a closure <code>calculate_winding(alpha::Int) -> ComplexF64</code> that evaluates the local winding density centred at site <code>alpha</code> (1-indexed).

Quenched case (`quenched=true`): Uses the 2-term decomposition $\langle \alpha | \hat{W} | \alpha \rangle = \Lambda [\cos(x_\alpha/\Lambda) \langle \alpha | W_1 | \alpha \rangle - \sin(x_\alpha/\Lambda) \langle \alpha | W_2 | \alpha \rangle]$ where $W_1 = \sigma_z(P \sin \hat{X} Q + Q \sin \hat{X} P)$, $W_2 = \sigma_z(P \cos \hat{X} Q + Q \cos \hat{X} P)$.

Flat case (`quenched=false`): builds $W = \sigma_z(P \hat{x} Q + Q \hat{x} P)$ once; the closure simply evaluates $\langle \alpha | W | \alpha \rangle$.

2D Real-Space Chern Marker

Signature	Description
<code>get_C_op_MP0_from_P(P, L, sites, xfunc, yfunc; l=nothing, Λ=10, maxdim=500, cutoff=1e-8, quenched=true) → Function</code>	Build real-space Chern marker from pre-computed projector P . Returns closure <code>calculate_chern_number(alpha::Int) → ComplexF64</code> .
<code>get_C(H, xfunc, yfunc; method=:KPM, fermi=0.0, l=nothing, Λ=10, Nchebychev=300, maxdim=500, cutoff=1e-8, Nel=nothing, quenched=true) → Function</code>	High-level wrapper: calls <code>_get_projector</code> then <code>get_C_op_MP0_from_P</code> . Returns same <code>calculate_chern_number</code> closure.

Quenched case (`quenched=true`): uses the 4-term decomposition

$$\langle \alpha | \hat{C} | \alpha \rangle = 2\pi i \Lambda^2 \left[\cos \frac{x_\alpha}{\Lambda} \cos \frac{y_\alpha}{\Lambda} \langle \alpha | C_1 | \alpha \rangle + \sin \frac{x_\alpha}{\Lambda} \sin \frac{y_\alpha}{\Lambda} \langle \alpha | C_2 | \alpha \rangle - \cos \frac{x_\alpha}{\Lambda} \sin \frac{y_\alpha}{\Lambda} \langle \alpha | C_3 | \alpha \rangle - \sin \frac{x_\alpha}{\Lambda} \cos \frac{y_\alpha}{\Lambda} \langle \alpha | C_4 | \alpha \rangle \right]$$

where C_1 – C_4 are pre-computed α -independent MPOs. Only scalar trig evaluations are needed per site.

Flat case (`quenched=false`): $\hat{C} = 2\pi i (Q\hat{x}P\hat{y}Q - P\hat{x}Q\hat{y}P)$.

DMRG (DMRG_tk.jl)

Signature	Description
<code>dmrg_gs(H, sites; nsweeps=10, maxdim=[10,20,50,100,200], cutoff=1e-8, noise=[...], linkdim_init=10) → (E, ψ)</code>	Ground state and energy of H via DMRG.
<code>build_K(H_mpo, sites, ω, η; maxdim_K=200, cutoff_K=1e-8) → MPO</code>	Resolvent-squared MPO $K(\omega, \eta) = (H - \omega I)^2 + \eta^2 I$. Ground-state energy $E_K \geq \eta^2$; $E_K \rightarrow \eta^2$ at spectral peaks.
<code>dmrg_spectral(H_mpo, sites, ω, η; ψ₀=nothing, ...) → (E_K, ψ)</code>	DMRG ground state of $K(\omega, \eta)$. Output $ \psi(\omega)\rangle$ concentrates on the eigenstate of H nearest ω .
<code>local_weight(ψ, i, L, sites) → Float64</code>	$ \langle i \psi \rangle ^2$: LDoS proxy at site i (0-indexed, big-endian). Use with <code>dmrg_spectral</code> output.

Time Evolution (Timeev_tk.jl)

Signature	Description
<code>tdvp_evolve(H, ψ, dt; maxdim=200, cutoff=1e-10, normalize=true, nsite=2) → MPS</code>	Single TDVP step; thin wrapper around <code>ITensorMPS.tdvp</code> .

<code>build_tdvpropagator_mpo(H, dt, L, sites; maxdim=50, cross_tol=1e-8, ...) → MPO</code>	Compress $U(\Delta t) = e^{-iH\Delta t}$ into an MPO via QTCI on matrix elements $\langle i U j \rangle$ evaluated by TDVP. Diagonal pivots seeded by default.
---	--

Random Phase Approximation (RPA.tk.jl)

Computes the RPA susceptibility χ^{RPA} via the Dyson equation $(1 - \Pi_0 V)\chi = \Pi_0$, where Π_0 is the Lindhard (bubble) function.

Pipeline

1. `get_Tnlists` — build Chebyshev lists for system 1, system 2, and the effective combined Hamiltonian $H_{\text{eff}} = \mathbf{I} \otimes H_2 - H_1 \otimes \mathbf{I}$.
2. `get_bubble_expanded_from_Tn` — compute $\Pi_0(\omega)$ as a $2L$ -site MPO.
3. `build_bubble_mpo` — collapse to an L -site MPO.
4. `rpa_from_bubble_diag` — solve the Dyson equation via DMRG linear solver.

Functions

Signature	Description
<code>get_Tnlists(H, H2, sites, sites2, N; a=6, maxdim=100) → (Tn1, Tn2, Tn_eff)</code>	Build 3 Chebyshev lists: individual and effective.
<code>get_bubble_expanded_from_Tn(Tn1, Tn2, Tn_eff, s1, s2, N, ω, ε_F; a=6, maxdim=200) → MPO</code>	Lindhard bubble $\Pi_0(\omega)$ as a $2L$ -site MPO.
<code>build_bubble_mpo(ω; Tn1, Tn2, Tn_eff, sites, sites2, finalsites, finalfinalsites, ...) → MPO</code>	Collapse $2L$ -site bubble to L -site MPO.
<code>rpa_from_bubble_diag(Π, V, finalsites, finalfinalsites; nsweeps=20, maxdim=400, cutoff=1e-8) → MPS</code>	Solve $(I - \Pi_0 V)\chi = \Pi_0$ via DMRG linear solver.

Tensor Utilities (used internally by RPA)

Signature	Description
<code>mpo_kron(A, B) → MPO</code>	Concatenate two MPOs (Kronecker product on site space).
<code>mps_kron(A, B) → MPS</code>	Concatenate two MPS.
<code>replace_sites(MPO, newsites) → MPO</code>	Remap physical indices of an MPO to <code>newsites</code> , preserving prime levels.
<code>interleave_mpo(mpo, phys_sites, n) → MPO</code>	Embed L -site MPO into $2L$ -site space by interleaving with identity.
<code>collapse_mpo_pairs(mpo2L, out_sites) → MPO</code>	Merge consecutive site pairs $(2n-1, 2n)$ to single site: $2L \rightarrow L$.
<code>extract_diagonal_to_mps(M) → MPS</code>	Extract diagonal of an MPO as an MPS by projecting <code>bra=ket</code> .

<code>merge_mps_to_mpo(mps) → MPO</code>	Contract consecutive pairs of an MPS into MPO tensors.
<code>apply_interleave_swaps(W, sites; cutoff=1e-16, maxdim=200) → MPO</code>	Reorder sites from block order $[1 \dots N, N+1 \dots 2N]$ to interleaved via SWAP gates.

Self-Consistent Mean Field (Meanfi.tk.jl)

Signature	Description
<code>SCF_Hubbard(L, sites, U, k_mpo, max_iter, N, threshold, mpo_guess_up, mpo_guess_dn, mps_guess_up, mps_guess_dn, qtt_den_old_up, qtt_den_old_dn, mix)</code>	Hartree–Fock SCF loop for the Hubbard model. Iterates $H_{\uparrow} = H_{\text{kin}} + U\rho_{\downarrow}$ and vice versa until $\ \rho_{\text{new}} - \rho_{\text{old}}\ < \text{threshold}$. Accepts linear mixing parameter <code>mix</code> . Returns <code>(qtt_new_up, qtt_new_dn, Tn_up, Tn_dn, conv_error)</code> .

Utilities (utils.jl)

Signature	Description
<code>to_binary_vector(n, L) → Vector{String}</code>	Convert integer n to length- L big-endian binary string vector.
<code>binary_to_MPS(n, L, sites) → MPS</code>	Computational-basis state $ n\rangle$ as an L -site MPS.
<code>matrix_checker(mpo, sites, i, j) → Number</code>	Matrix element $\langle i \text{mpo} j \rangle$ by contracting basis MPS (small systems only). Optional <code>L</code> argument accepted but ignored.
<code>get_matrix(mpo, sites) → Matrix</code>	Full $D \times D$ dense matrix of <code>mpo</code> (feasible for $D \lesssim 512$). Optional <code>L</code> argument accepted but ignored. Works for any site types.
<code>fix_sites(mpo, sites) → MPO</code>	Replace site indices of an MPO (e.g. from TCI) with target <code>sites</code> .
<code>custom_mpo(mps, new_sites) → MPO</code>	Convert a $2N$ -site MPS (interleaved QTCI) into an N -site MPO.
<code>fused_mpo(mps, new_sites) → MPO</code>	Convert an N -site MPS with dim-4 physical indices (fused grid) into an N -site MPO.
<code>mps2mpo(L, sites, density_mps) → MPO</code>	Promote a diagonal MPS to a diagonal MPO via <code>_asdiagonal</code> .
<code>custom_mps(qtt, sites) → MPS</code>	Replace site indices of a TCI MPS with target <code>sites</code> .

Custom Operator Extensions

Defined in `utils.jl` and `2D_lattice.jl`:

Name	Site type	Matrix
<code>sigma_plus</code>	Qubit	$\begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$ — binary increment (up-shift)
<code>sigma_minus</code>	Qubit	$\begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}$ — binary decrement (down-shift)
<code>sigma_d</code>	Qubit	$\begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}$ — projector onto $ 1\rangle$
<code>sigma_u</code>	Qubit	$\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$ — projector onto $ 0\rangle$

Built-in $\text{spin-}\frac{1}{2}$ operators (used by `prepend_spin/postpend_spin`):

Symbol	Matrix	Symbol	Matrix
<code>:Id</code>	$\mathbf{I}_2$	<code>:Sz</code>	$\frac{1}{2}\sigma_z = \text{diag}(\frac{1}{2}, -\frac{1}{2})$
<code>:Pup</code>	$\text{diag}(1, 0)$	<code>:Sp</code>	$ \uparrow\rangle\langle\downarrow $
<code>:Pdn</code>	$\text{diag}(0, 1)$	<code>:Sm</code>	$ \downarrow\rangle\langle\uparrow $
<code>:Sx</code>	$\frac{1}{2}\sigma_x$	<code>:Sy</code>	$\frac{1}{2}\sigma_y$
<code>:iSy</code>	$i\sigma_y = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$	<code>:miSy</code>	$-i\sigma_y$

Built-in Nambu operators (used by `prepend_nambu/postpend_nambu`):

<code>:Id</code>	$\mathbf{I}_2$	<code>:tz</code>	$\tau_z = \text{diag}(1, -1)$
<code>:Pp</code>	$\text{diag}(1, 0)$	<code>:tp</code>	$ p\rangle\langle h $
<code>:Ph</code>	$\text{diag}(0, 1)$	<code>:tm</code>	$ h\rangle\langle p $
<code>:tx</code>	σ_x	<code>:ty</code>	σ_y

Quick-Reference Workflows

KPM Density of States and Band Structure

```
# 1. Build Hamiltonian (scale=0 means lazy DMRG estimation)
H = get_Hamiltonian("square_2d", 1.0; L=8)

# 2. Build Chebyshev list (caches in H._tn_cache / H.scale / H.center)
Tn, scale, center = KPM_Tn(H, 200; maxdim=100)

# 3. Sweep in energy (always rescale before calling KPM functions)
omega_r = (omega_phys - center) / scale # must be in (-1, 1)

# 4. Observables
A = get_ldos_w_from_Tn(Tn, 200, omega_r; maxdim=100) # (H)
dos = real(tr(A)) # density of states
G = get_Green_retarded_from_Tn(Tn, 200, omega_r; eta=0.02)
rho = get_density_from_Tn(Tn, 200; fermi=omega_r_fermi)

# 5. Band structure A(k,) | works for plain, spinful, BdG, layered H
Akw = get_bands(H, omega_phys_vals) # trace over aux DOF
Akw = get_bands(H, omega_phys_vals; aux_proj=[1]) # spin-up only
```

Density Matrix Purification

```
# McWeeny: one-liner (initial guess built and cached internally)
rho_mc = mcweeny_purify(H; maxiters=30, maxdim=200, tol=1e-5, verbose=true)

# SP2: one-liner with explicit electron count
rho_sp = sp2_purify(H; Nel=H.N÷2, maxiters=40, maxdim=200)

# Low-level form (for non-standard center or external MPO):
rho0 = purification_initial_guess(H; maxdim=100, cutoff=1e-8)
rho = mcweeny_purify(rho0; maxiters=30, maxdim=200)
```

Topological Invariants

```
# Define coordinate functions (0-indexed site i, 0-indexed length L_chain)
xfunc(i, L) = Float64(i % Int(sqrt(L))) # x-coordinate on square lattice
yfunc(i, L) = Float64(i ÷ Int(sqrt(L))) # y-coordinate
sz_func(i) = isodd(i) ? 1.0 : -1.0 # sublattice / sigma_z (1-indexed)

# 1D winding number (returns a closure; pre-computes 2 MPOs once)
winding = get_W(H, xfunc, sz_func; quenched=true, =10.0, maxdim=40)
nu = sum(real(winding(alpha)) for alpha in 1:H.N) / H.N

# 2D Chern marker (returns a closure; pre-computes C1{C4 MPOs once)
chern = get_C(H, xfunc, yfunc; quenched=true, =10.0, maxdim=500)
C_local = real(chern(H.N ÷ 2)) # evaluate at central site
C_avg = sum(real(chern(a)) for a in 1:H.N) / H.N
```

Bilayer and Twisted Multilayer

```
# Bernal bilayer graphene (exact interlayer, no TCI)
H_bg = bilayer_hamiltonian(:honeycomb, 4, 3;
    stacking=:Bernal, t_intra=1.0, t_inter=0.3)
# H_bg.sites = [layer_s, pos_qubits...]

# Twisted bilayer at 5 degrees
H_tbg = twisted_bilayer_hamiltonian(:triangular, 4, 4, 5.0;
    t_intra=1.0, t_inter=0.3, _decay=1/16)
```

Spinful Superconductor (BdG)

```
H = get_Hamiltonian("square_2d", 1.0; L=8)
add_onsite!(H, _ -> -2.0) # chemical potential
add_zeeman!(H, 0.1; direction=:z) # spin-splitting
add_soc!(H, 0.05; type=:rashba) # Rashba SOC
add_superconductivity!(H, 0.3) # singlet pairing
# H.sites = [nambu_s, spin_s, pos_qubits...] (aux_side=:pre)
# H.N is unchanged; the MPO now spans 2×2×N = 4N states

# Band structure tracing over Nambu and spin:
Akw = get_bands(H, omega_phys_vals)
# Band structure in the particle sector only:
Akw = get_bands(H, omega_phys_vals; aux_proj=[1, 1])
```

aux_side: postpending auxiliary DOF

By default (`aux_side=:pre`) auxiliary indices are prepended. To instead append them, pass `position=:post` to the first extension function:

```
H = get_Hamiltonian("square_2d", 1.0; L=8)
add_spin!(H; position=:post)           # H.sites = [pos_qubits..., spin_s]
add_superconductivity!(H, 0.1)        # H.sites = [pos_qubits..., spin_s, nambu_s]
# All subsequent add_* calls inherit H.aux_side = :post
```